```
          },
        ...
    };
    return () => {
      delete window.onYouTubeIframeAPIReady;
    };
  }, []);
  ...
};
```

Jane’s solution reflected a strong grasp of TypeScript’s type augmentation mechanisms, modular and global type distinctions, and the importance of maintaining type safety when interfacing with dynamic or external JavaScript APIs. She also demonstrated adaptability by providing multiple solutions and evaluating their trade-offs.

3.11 TypeScript: Current trends and future outlook

TypeScript has firmly established itself as a cornerstone of modern web development. Let’s explore some current trends and future directions that candidates should be aware of to demonstrate up-to-date expertise in technical interviews.

Traditionally, TypeScript required transpilation into JavaScript using tools like `tsc`. However, inspired by Deno (https://deno.com) and Bun (https://bun.sh), which offer first-class TypeScript support out of the box, modern advancements in Node.js now allow developers to run TypeScript files natively without manual transpilation.

Node.js v22.6.0 introduced experimental support for "type stripping" via the `--experimental-strip-types` flag.

```
node --experimental-strip-types example.ts
```

Node.js v22.7.0 added support for transforming TypeScript-only syntax (e.g., enums) with the `--experimental-transform-types` flag.

```
node --experimental-transform-types example.ts
```

Starting from Node.js v23, the `--experimental-strip-types` flag is enabled